

Risks of Agile Software Development: Learning from Adopters

Amany Elbanna, Royal Holloway University of London, UK, email:
amany.elbanna@rhul.ac.uk

Suprateek Sarker, University of Virginia, USA & Royal Holloway University of London, UK,
email: sarkers@virginia.edu

Abstract:

Agile software development (ASD) emerged as a result of widespread professional discontent with traditional approaches, the high failure rates associated with software development projects, and most importantly, the need for speedy responsive delivery of software particularly for Internet businesses. Today, ASD approaches are embraced by many practitioners, and often extolled by commentators as being ‘faster, better, cheaper’ approach to software development. However, a recent high profile failure of an ASD project and the discontinuance of the use of ASD practices in a software company have drawn attention to possible risks associated with ASD. Before we can manage such risks, an important first step is to *identify* the risks and understand *why these risks arise* in the ASD context. In this paper, we offer insights derived from the examination of ASD practices in 28 organizations. Based on 112 interviews conducted in these organizations and further 25 interviews conducted with agile software development contractors (developers and project managers) and consultants, we unearthed key risk factors related to ASD practice that need to be carefully managed to achieve desired project outcomes.

Keywords: Agile software development, Agile risks, software development risks, software risk management

1. INTRODUCTION

Agile software development (ASD) approaches continue to grow in popularity as organizations increasingly seek speed in developing software and the capability to cope with changes in the current fast moving business environment. ASD is an umbrella term that encompasses a variety

of approaches and methods that have been developed by software practitioners, including Extreme Programming (XP), Scrum, the Dynamic Systems Development Method (DSDM), Crystal, Agile Project Management (APM), Feature Driven Development (FDD), and Lean Software Development (LSD). These approaches may differ in specific terminology and practices; however, they share many of the core values presented in the Agile Manifesto¹ [1]. In general, they focus on providing a software development approach that is responsive to both customers' needs and the needs of developers. They also share a general orientation towards short "time-boxed" iterative development cycles, frequent communication with customers, and constant adaptation, and accommodation of change.

Recent failures of projects adopting agile approaches to software development and project management, and reported challenges associated with agile approaches [2, 3] have drawn attention to the possible existence of risks associated with agile development and project management, and the importance of identifying, assessing, and mitigating these risks. For example, in the UK, the Department for Work and Pensions has, in 2013, ceased all software development on its originally planned £2.2bn Universal Credit program after the cost escalated to nearly £12.8bn, six times the originally planned figures and the project failed to deliver [4]. Universal Credit is one of the largest agile software development projects in the UK, and hence one might dismiss the unacceptable outcome as an exception. One might also dismiss the outcome on suspicion that agile could have been used as a scapegoat for other project management's problems. However, the project continues to draw attention to the possibility of having agile-related risks and the need to identify them. The recently reported withdrawal of agile practices (Scrum) from a software development company [5] is yet another case that highlights the existence of risks that need to be managed.

ASD approaches change how the software organization works (in terms of processes and teams organization) and the nature of its relationship with the business organization. Such change has produced desirable effects and has positively resolved many of previously identified software development risks. However, as was experienced with the introduction of many other

¹ Manifesto, A., <http://agilemanifesto.org>, 2001

process innovations such as business process reengineering in organizations, the introduction of agile approaches in software development teams can lead to both desirable and undesirable effects. Interestingly, a study on ASD critical success factors showed that of 12 typical IT critical success factors, only half were supported by ASD approaches with different degrees of strength. [6]. This suggests that risk factors (and success factors) relevant to ASD projects are different than those applicable to traditional software development environments. Identifying and managing risks is one of the important areas of research and good practice in software development, wherein risk identification and analysis is seen as the first step in reducing the chance of project failures [7]. Given the differences between traditional and ASD approaches, we believe the identification of ASD-related risks needs a fresh view, that is not overly reliant on our previous knowledge of (traditional) software development risks *alone*. Past research has identified a number of risk factors. Alter and Sherer (2004) note that many of the risk factors that apply to operations of IS also apply to IS projects [8]. Studies of IS risk have examined project and operational risk in general. The highlighted risks are typically related to users such as poor communication with users and stakeholders, lack of user involvement, failure to manage end-user expectations, misunderstanding of user requirements and failure to accommodate change in requirements and scope; these issues are found to be effectively managed through the adoption of ASD approaches. For example, the short iteration cycles allow for accommodating change and clarifying users' requirements while the face-to-face interaction and constant communication with users and business stakeholders improves communication and develops common understanding of requirements [9]. However, the *specific* risks related to ASD remain unexplored.

As a first step, this study seeks to identify these risks and the different practices of dealing with them². Due to relative paucity of research in this area and in order to provide a fresh view, away from typical software development risks, we turn to the people and organizations involved in

² This paper is a significantly enhanced version of a paper "Identifying the risks associated with agile software development: an empirical examination," *Mediterranean Conference in Information Systems* 2014 (MCIS), Verona, Italy, authored by Amany Elbanna. While the earlier version focused on the dynamics between task, structure, people and technology that caused gaps in the business, IT, and operational levels, this paper focuses on the practices in terms of identifying and dealing with risks related to agile software development.

ASD, to learn from their practices and experiences. It is worth emphasizing that the study does not provide statistical testing or a comparison between the theory of ASD and the actual practices in organizations. *It aims to provide insight based on participants' views of practice.* Future research should take these findings and subject them to empirical testing using large, well-designed samples as previous research in software development risks did decades ago. However, this paper offers an important foundation for such follow-up research. Our study is guided by the following research questions:

- *What are the key risks related to agile software development (ASD) approaches that are faced by adopters of the approaches?*
- *Why do these risks arise and what are their consequences?*
- *Finally, how do different organizations deal with these risks?*

How the study was conducted

This study adopts a qualitative, interpretive approach which is particularly useful when researchers seek to understand people's experience. In this case, we seek to gain insight from the experiences of the different stakeholders of projects using agile approaches that is difficult to capture using experience-far approaches such as simulation, modeling, or surveys. The study, initiated in 2008, is part of an ongoing funded research effort on the adoption of agile software development (ASD). Data collection was primarily done through interviews, a well-established research method. Like any method, interviews have been associated with many strengths (e.g., in-depth, holistic coverage, being close to practice, opportunity to discover patterns not previously known) and weaknesses (e.g., bias introduced due to selection of interviewees, faulty recall, social desirability of responses, and political agenda of different respondents). We used established guidelines to minimize these limitations. The results described in this article are based on an investigation of the adoption of agile software approaches in *28 different organizations*, of which 4 were examined longitudinally over multiple years. Members of software development teams, project managers, IT directors, business managers and users were interviewed. The total number of corporate interviews was *112 interviews*. 39 interviewees agreed to have their conversations audio recorded, and these interviews could be transcribed verbatim. As per common practice, in cases where interviewees refused to have their interviews recorded, the researchers took notes during the interviews, and when feasible, at the end of the interview, presented a summary of the conversation and key points to the interviewee for

validation/ clarification. Data was analyzed over different cycles³. The first cycle coding resulted in the identification of key risks experienced by participants in the projects. In the second cycle, the identified risks were grouped according to their nature and origin. These were then refined through cross-case comparisons. The risks reported here are those found to be prominent in at least four different cases; those found to be present in three cases or less are not reported in this paper due to the nature of the article and space constraints. Table 1 exhibits the number of companies in each sector and the purpose of the software development efforts. In addition to stakeholders within the companies, 17 freelance developers and 8 consultants working on or leading 8 different consulting firms on ASD approaches were also interviewed. Also, the first author has attended 9 business-oriented conferences and workshops focusing on ASD from 2007-2013, 3 of which as a speaker. In these meetings, she presented parts of the findings of this study and had conversations with other participants. Interviews with participants with different perspectives on the projects and triangulating the interviews helped assess reliability and enhance the validity of the findings. In addition, the conversations with and feedback from workshop participants, who were not themselves involved with the studied organizations, helped ensure the face validity as well as the external validity of the findings.

Table 1: An overview of the studied organizations

Sector	Total no. of companies	Internal use	External customer	Commercial	Internal & Commercial
Utility	2	1		1	
Transportation	3	3			
Telecommunication	3			2	1
Media	2	2			
Financial services	7	7			
Software company	9		7		2
Local administration	1	1			
Travel and Leisure	1	1			
Total	28	15	7	3	3

³ Miles, M., A. Huberman, and J. Saldana, *Qualitative Data Analysis: A Methods Sourcebook*. 3rd ed. 2014: Sage publications.

2. THE FINDINGS

Previous research has shown that ASD approaches can speed the delivery of software, and increase business satisfaction. In our research, we find evidence for the same, and, thus, we recognize the many significant benefits that ASD adoption can lead to. However, our research also highlights some of the potential risks involved in the adoption of ASD, some of which have not been highlighted sufficiently in the literature. Being aware of these possible risks is an important first step towards their effective management. Below, we discuss the key software and management-related ASD risks identified. Table 2 presents a summary of these risks. It also shows the number of organizations where these risks were evident in our study, and the ways different organizations are dealing with these risks.

Table 2: Key agile software development (ASD) risks in the studied organizations⁴

Key risks	Number of organizations experiencing these risks (out of 28 organizations studied)	Typical organizational approaches of dealing with risks, with different levels of effectiveness
<i>Development and deployment risks</i>		
<i>Technical debt</i>	23	- make lists of outstanding technical issues and convince the business to invest in allocating one or more iteration cycle(s) for addressing them. - address only the most important technical issues that are short-term and could immediately impact users, and ignore others. - under business pressure, ignore accumulated technical debt and continue development till issues surface and require attention.

⁴ Descriptions of the risks such as technical debt are provided in the text following the table.

<i>Fractured development and operation</i>	21	- one member of the ASD team has to be from operations. - invite operation teams to ASD planning sessions and end of iteration demonstrations. - allocate an extra week (in fact, in a highly sensitive systems in a large financial services firm, two weeks were allocated to a typical two-weeks iteration cycle) in each iteration cycle to address operational issues⁵.
Increased defects in newly formed teams	13	- allocate experienced senior developers to agile teams - allocate at least one experienced software developer to each agile team - provide on-the-job training and support to deal with software developers' uneasiness with ASD particularly those who lack previous ASD experience.
Project Management Risks		
<i>Fragmentation of project management tools</i>	17	- audit all existing agile development teams for project management and software testing tools in use and discuss standardization. - negotiate limiting the number of tools in use while continue to accept some variations. - not seen as a problem as long ASD teams delivers on target. - Specify standard tools and request all teams to comply. However, this approach is resisted by ASD teams who value autonomy.
<i>Knowledge Retention</i>	20	- Use wikis to document project progress and explain key decisions. - Create and use software code libraries and encourage code re-use. - Document important technical specifications and key technical decisions. - Encourage the use of social media among ASD team members and across teams.

Development and deployment risks

Various risks that could influence the quality and operations of the developed software were identified. The most prominent risks that require mitigation are as follows.

⁵ Vidgen and Wang (2009) analysis and findings also show that each team needs to find its own pace in each specific context. Secondly, a team needs to understand what pace can be sustained over time. A suitable pace strikes a balance such that the iteration cycle is long enough to get some meaningful work done but short enough not to lose momentum and responsiveness to change [15].

Technical debt: This is defined as the visible and invisible results of past decisions and short-term compromises about a software that creates complexities and could negatively affects its future [10]. This concept was originally used in reference to coding practices and has now been extended to include architectural, testing, or documentation debt. While technical debt was initially discussed in the context of traditional software development [11], in ASD, technical debt concerns can be intensified. In ASD, the debt can quickly accumulate due to the need to: significantly reduce development time, adhere to strict time-boxing, and deliver functional requirements for business use constantly. Incurring and accumulating technical debt could lead to unexpected project delays⁶, lower software quality in the long run as the software becomes more complex and less understandable and thus more difficult to maintain. Risks of security breaches and data corruption are also enhanced, if security debts are being accumulated⁷. The technical teams we studied showed understanding of the dangers of letting technical debt accumulates; however, they also explained that the pressure of delivering in short time-boxed iteration cycles made them take “shortcuts” to speed up delivery⁸. Furthermore, they highlighted the difficulties faced in logging in non-functional requirements as users’ stories in iteration cycles, and how the business-oriented stakeholders often deprioritized these requirements⁹. This could be attributed to the lack of comfort of business participants with “technical jargon” and also to their single-minded focus to push on with the implementation of functional requirements.

⁶ Cunningham (1992), who coined the metaphor of technical debt, has asserted that “A little debt speeds development so long as it is paid back promptly with a rewrite. The danger occurs when the debt is not repaid. Every minute spent on not-quite-right code counts as interest on that debt”. Cunningham, W. 1992. The WyCash Portfolio Management System. *OOPSLA’92 Experience Report*.

⁷ Security debt is often accumulated because critical and high vulnerabilities are attended to conscientiously while medium and low vulnerabilities are ignored or kept for later consideration, leading to the accumulation of such debt. It should be noted that it is generally impossible to have zero security debt; the objective should be to identify and formulate a repayment plan.

⁸ This is in agreement with Cohn (2010) who also states that “Technical debt is often the result of a rushed implementation.” Cohn, Mike. *Succeeding with agile: software development using Scrum*. Pearson Education, 2010, p. 321.

⁹ This could offer explanation to Stray et al. (2013) findings that daily Scrum meetings were used in their studied organization to justify the continuation of investing in a failing course of action leadings to many project’s setbacks. Stray, V. G., Moe, N. B., & Dybå, T. (2012). Escalation of commitment: a longitudinal case study of daily meetings. In *Agile Processes in Software Engineering and Extreme Programming* (pp. 153-167), Springer Berlin Heidelberg.

In summary, the respondents felt that while the close involvement of business stakeholders as part of ASD was helping to mitigate some of the risks identified with traditional software development (e.g., poor communication with users and stakeholders, lack of user involvement, failure to manage end users expectations), it was also significantly contributing to their projects' technical debt in many instances¹⁰.

Fractured development and operation: This refers to the separation between the software development teams and the IT operations teams. As business and software development teams are brought close together in the ASD environment, the work of the software development teams becomes tightly coupled with business needs, requirements, and timeline. However, the development work becomes increasingly separated from IT operations teams. IT operations teams include systems administrators, network engineers, and infrastructure specialists who ensure the stability and reliability of the software in its operational environment. This separation between the software development and operation teams is amplified in ASD due to the difference of the work practices and pace of each side. The impact of this separation is further described below.

1) *Tension in the relationship between development and operations.* This tension between the development and operations teams typically surfaces as ASD teams find operations' response to be "too slow" while operations teams find ASD teams' push for continuous delivery to be too "stressful" as they (i.e., operations teams) try to cope with their provisioning of applications and infrastructure tasks. The reason behind this conflict is that the pace of ASD approaches challenges the work rhythm and priorities of the operations teams. For example, ASD teams work and deliver within short iteration cycles, demanding much parallelism in work and roles, which is inconsistent with the work of operations teams which function in a rather linear way, based on logging of requests in queues and responding to them in a systematic order. Hence, ASD teams' operations requests are typically met in different iteration cycles than required.

2) *Agile software development (ASD) teams not fully aware of the operations environment.* The lack of communication with and close involvement of operations in ASD environment means

¹⁰ These are the views expressed by the participants of the study. It should be noted that in general not appreciating or understanding the importance of something could be because either the source (IT staff) do not communicate well, or the receiver (Business staff) could not see the value.

that software development and testing are conducted in environments that could be different from the actual operating or production environment. For example, in one of the studied organizations, the software development was done using a version of a particular database technology, and it was later discovered that a different version was being used in the production environment. As a consequence, the application could not be implemented as per schedule, and considerable re-work had to be undertaken to prepare the application for the current version of the operating environment.

ASD environment lacks the mechanisms to deal with deployment and operability issues. The inability of ASD teams to involve operation teams can cause delay in deployment and production. One of the organizations we studied attempted to solve this problem by dedicating a sprint to operation issues and planned it as a major user story in the project backlog early in the project. Another organization made it a requirement for teams to involve at least one team member from operations in each ASD project for the entire project duration.

3) *Lack of knowledge of deployment and operability.* As technology advances, software developers do not necessarily have the required operational knowledge and experience. The increasing use of off-the-shelf components, particularly in web development, requires specialists to operate them. Web development includes components such as web servers, application containers, networking, and databases that are not integrated into the software; yet the need to be configured and maintained which required considerable effort. These tasks are usually beyond many software developers' capabilities and hence need to be carefully overseen by ASD team management. While this is the case in most software development environments, the ASD environment accentuates this risk by narrowing the development teams' focus on the delivering of working software with desired business functionality. Testing, especially unit testing, test-driven delivery, and User Acceptance Testing, is geared to ensure the standard of functional requirements and general performance of the software.

4) *Increased firefighting:* Operations teams expressed the view that the way ASD teams conducted work necessitated much "firefighting" on the operations side in order to stabilize the systems, and make them available for use. ASD teams struggle to balance the development of new features and handling the project backlog, i.e., dealing with re-work requested either in defects reports or enhancements requests. In fact, it was reported that ASD teams, more often than not, tend to focus on developing new features at the expense of attending to re-work

requests. This can lead to emergency patches and significantly more work downstream for ASD as well as operations teams.

5) *Business push for new features.* Infrastructure and networking operational issues are necessary for the system to operate in its live environment. However, with the focus on the business need and meeting of user stories and business-led priorities, these important technical issues such as servers or network upgrade are not given sufficient priority by the business; hence, they do not get included in the iteration cycles until the end, when ASD teams start thinking of the going live phase – this can cause unanticipated delays in the overall project. A team member expressed his disappointment as follows “I am afraid that the time we save in development is actually wasted in operations”.

Increased defects in newly formed agile software development (ASD) teams. In smaller organizations and especially in early adoption projects, it was highlighted that the number of defects had risen beyond the organizational average; the good news, however, was that the defect rates approached acceptable levels as the team matured. The experience and quality of developers were identified as more important in the ASD environment than in traditional environment. According to thirteen of the studied organizations, the quality of the product depends largely on the quality of developers and their experience in the ASD environment. Developers who lack previous ASD experience tend to perform poorly in their first ASD projects. Reflecting on her early days with ASD, a developer recalled her frustration: “When I came here, they were doing agile, I found myself spending my time talking and not actually doing it [development]... now I can see it works”. Another member of a team that was using Scrum for the first project, reflected that “thinking of the project I’m on at the moment ... we had to put in complete defect management around it... as an after-thought”.

Project management risks

Various risks that could affect the management of teams and projects were identified. Some of the prominent risks requiring careful mitigation are as follows.

Fragmentation of project management tools. ASD approaches advocate self-organized teams. This offers ASD teams considerable freedom and autonomy, and can be motivating for many team-members. However, in their attempts to develop and organize their own work routines, ASD teams tend to select and use project management and testing tools based on their own preferences. From the organizational perspective, this carries the risk of fragmenting the used tools across the enterprise, losing the economies of scale related to licensing of tools, and also increasing the complexities of integrating work across domains and teams.

For example, in nineteen of the organizations we studied, ASD teams had moved away from the traditional whiteboard and sticky-notes that agile software practices have been associated with in the past. Many developers reported that they found whiteboards and sticky-notes rather inconvenient as notes tended to fall down and lose their order. Others found it cumbersome to keep moving the sticky notes across the whiteboard. Moving away from the past concern of ‘which sticky note is the stickiest’ [12], they use agile project management software to organize their effort. The choice of such software largely depended on the developers’ knowledge and preferences. In many organizations, the lack of standardization regarding agile project management software led to the use of different software products, causing much confusion. Indeed, IT managers reported that merging teams and managing workloads became rather challenging in such conditions.

Another example pertains to the use of version control systems and to User Acceptance Testing. In one of the studied organizations ASD teams used 8 different version control systems and 4 User Acceptance Testing protocols. Being autonomous, ASD teams held the view that “I use what I am comfortable with, and they can use whatever they want and are happy with”. Apart from resulting in the loss of negotiation power of the company with tool vendors and the potential economies of scale in licensing, such fragmentation in tool use negatively impacted knowledge sharing across projects and teams, and was disorienting for the operations team-members, who found it difficult to find systems-related data.

Knowledge Retention. ASD often privileges face-to-face communication over written documentation. This implicitly assumes that members of the same team are retained until the

end of the project, and members of the development team of a particular system remain with the organization to oversee the development of the following versions of the application. In practice, these assumptions do not necessarily hold. Indeed, we found that in a multi-project environments, the shuffling and re-assignment of team members is common practice.

Our study also revealed that the team velocity and quality tended to drop noticeably with the joining of a new team member. In addition, the high turnover in IT personnel means that it is unlikely that all the development team would still be involved with subsequent versions of the application. Owing to the light documentation in most agile projects, and not having the same team that developed the application throughout its lifecycle, the subsequent versions of the application take longer to develop, with the new team trying to unravel the logic behind the code and to understand why certain development decisions were made in a past iteration.

3. Conclusion

The findings of this study¹¹ suggest that there are unique potential risks associated with the practices of ASD. While the practices of ASD can tackle some of the traditional risk factors such as lack of user involvement, misunderstanding of requirement, failure to gain users commitment, and change of scope, it also introduces new risks. The study identified and elaborated on some of the key risks that ASD adopters face and how they may be addressed (see Table 2). We submit that many of these risks are not self-evident, and thus should be consciously assessed and carefully managed throughout the life of ASD projects. It is worth noting that the development and operation silo has been recognized by the DevOps movement as members of this community try to find techniques to consistently address this problem [13]. The tension between ASD and other organizational functions such as quality assurance and strategic alignment has also been alluded to in some recent studies [14].

This research offers a critical look at ASD with a discussion of potential pitfalls. While previous studies have examined the benefits and the adoption of ASD, our study attempts to unearth notable risks, and their consequences. In doing so, this paper offers a word of caution and

¹¹ As noted earlier, we reiterate that like other qualitative, interview-based studies, our study does not seek to provide statistically validated results, but offers in-depth understanding of key themes related to ASD from the perspective of practitioners.

highlights some issues to think about for organizations as they decide whether or not to, and how to, embrace agile approaches.

References

1. Conboy, K., *Agility from first principles: reconstructing the concept of agility in information systems development*. Information Systems Research, 2009. **20**(3): p. 329-354.
2. Tessem, B., *Experiences in learning xp practices: A qualitative study*, in *Extreme Programming and Agile Processes in Software Engineering*. 2003, Springer. p. 131-137.
3. Rosenberg, D. and M. Stephens, *Extreme programming refactored: the case against XP*. 2003, Berkeley, CA: Apress.
4. Ballard, M., *Why agile development failed for Universal Credit*. Computer Weekly, 2013. **4 July 2013**: p. <http://www.computerweekly.com/news/2240187478/Why-agile-development-failed-for-Universal-Credit>.
5. Ralph, P. and P. Shportun, *Scrum Abandonment in Distributed Teams: A Revelatory Case*, in *PACIS 2013 Proceedings. Paper 42*. <http://aisel.aisnet.org/pacis2013/42>. 2013.
6. Chow, T. and D.-B. Cao, *A survey study of critical success factors in agile software projects*. Journal of Systems and Software, 2008. **81**(6): p. 961-971.
7. Schmidt, R., et al., *Identifying software project risks: an international Delphi study*. Journal of management information systems, 2001. **17**(4): p. 5-36.
8. Sherer, S.A. and S. Alter, *Information system risks and risk factors: are they mostly about information systems*. Communications of the Association for Information Systems, 2004. **14**(2): p. 29-64.
9. Baskerville, R. and H. Pries-Heje, *Short Cycle Time Systems Development*. Information Systems Journal, 2004. **14**: p. 237-264.
10. Kruchten, P., R.L. Nord, and I. Ozkaya, *Technical debt: from metaphor to theory and practice*. IEEE Software, 2012. **29**(6): p. 18-21.
11. Lim, E., N. Taksande, and C. seaman, *A Balancing Act: What software practitioners have to say about technical debt*. IEEE Software, 2012. **November/December**: p. 22-27.
12. Project Management Stack Exchange. *Which sticky note is the stickiest?* 2011 [cited <http://pm.stackexchange.com/questions/3219/which-sticky-note-is-the-stickiest>].
13. Spinellis, Diomidis. "Don't Install Software by Hand." Software, IEEE 29.4 (2012): 86-87.
14. Fitzgerald, B., et al. *Scaling agile methods to regulated environments: An industry case study*. in *Proceedings of the 2013 International Conference on Software Engineering*. 2013. IEEE Press.
15. Vidgen, R. and X. Wang, *Coevolving systems and the organization of agile software development*. Information Systems Research, 2009. **20**(3): p. 355-376.

Authors Biography



Dr Amany Elbanna is a senior lecturer at Royal Holloway University of London. She holds a MSc and PhD in information systems from The London School of Economics and Political Science. She is a member of the Association of Information Systems, UK Academy of Information Systems and IFIP WG 8.6 on adoption and diffusion of information systems. IT project and operation management is one of Amany's main research streams. The research on Agile software development is ongoing since 2007 and has been supported by a grant from the British Academy (SG-4835) and another from Loughborough University. Contact her at: amany.elbanna@rhul.ac.uk



Dr. Suprateek Sarker is a Professor at the McIntire School of Commerce, University of Virginia, and a part-time University of London Chair of Technology & Information management at Royal Holloway, UK. He also holds a Visiting Distinguished Professorship at Aalto University School of Business, Helsinki, Finland. Suprateek is currently serving as the Editor-in-Chief of the Journal of the AIS, a Senior Editor of Decision Sciences Journal, a Senior Editor (Emeritus) of MIS Quarterly, a member of the Board of Editors of Journal of the MIS, and an editorial board member of IEEE Transactions on Engineering Management and IT & People. He is a past recipient of the Stafford Beer Medal (with Prof. S. Sahay) awarded by the OR Society, UK.